Em Java, **metadados** são dados que descrevem outros dados do programa — ou seja, informações embutidas no código-fonte que não alteram a lógica direta de execução, mas explicam ou orientam como aquele código deve ser tratado. [1, 2]

Eles são implementados nativamente através das **Anotações (`Annotations`)**, introduzidas no Java 5 (`JSR 175`). [1, 2]

Para que servem?

As anotações (metadados) permitem adicionar informações complementares a classes, métodos, atributos, parâmetros e pacotes. Elas podem ser consumidas em três momentos principais: [1, 2, 3]

1. **Pelo Compilador:** O compilador pode usar o metadado para realizar verificações ou emitir avisos (ex.: `@Override` avisa que o método está sobrescrevendo um da classe pai; `@Deprecated` avisa que um método está obsoleto). [1, 2]
2. **Em Tempo de Compilação (Ferramentas de Geração):** Ferramentas externas ou processadores de anotações conseguem ler esses dados para gerar código adicional, arquivos de configuração ou documentação. [1, 2]
3. **Em Tempo de Execução (`Runtime`):** Frameworks (como Spring, Hibernate/JPA, Jackson) usam a **Reflection** para ler os metadados enquanto o sistema está rodando e tomar decisões dinâmicas. [1, 2]

Exemplo Prático

Um exemplo clássico é o uso no **Spring Framework** ou no **JPA/Hibernate**:

java

```java
@Entity
@Table(name = "usuarios")
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String nome;

    @Override
    public String toString() {
        return "Usuario [id=" + id + ", nome=" + nome + "]";
```

```
        }
    }
```

Use o código com cuidado.

- **`@Entity` e `@Table`:** Dizem ao Hibernate que esta classe é uma tabela do banco de dados chamada `usuarios`.
- **`@Id` e `@GeneratedValue`:** Indicam que o campo `id` é a chave primária e autoincremento.
- **`@Column(nullable = false)`:** Define que a coluna no banco não aceita valores nulos.
- **`@Override`:** Informa ao compilador que o método `toString()` está sobrescrevendo o método original da classe `Object`. [1]

Como são criados?

Você pode criar suas próprias anotações personalizadas utilizando a palavra-chave `@interface`: [1]

java

```
public @interface MinhaAnotacao {
    String autor();
    int versao() default 1;
}
```

Use o código com cuidado.

Você pode saber mais sobre a mecânica dessas estruturas no artigo detalhado da DevMedia sobre Entendendo Anotações em Java ou na visão geral do DIO - Entendendo Anotações em Java. [1, 2]